

CS120 Project Report

Authors

Bo Shen shenbo2022@shanghaitech.edu.cn

Yikun Pei peiyk2022@shanghaitech.edu.cn

Project 1

The primary objective of Project 1 was to employ sound as a medium for signal transmission, thereby enabling the realization of physical-layer functionality within a multi-device network. The project was developed using the JUCE framework, which extended and built upon the foundational components established in Project 0.

After thoroughly evaluating various technical factors such as bit error rate, the overhead associated with data fragmentation, and the challenges related to preamble detection, we ultimately decided on a packet structure where each packet carries 100 bits of payload data, preceded by a preamble of 440 bits. It is evident that this configuration results in a notably high ratio of preamble to data, which inevitably introduces considerable overhead into the transmission process. Although we attempted to reduce the preamble length in order to minimize this overhead, further reduction led to a significant and unacceptable decline in the preamble recognition rate during testing. Consequently, we chose to retain the 440-bit preamble as a necessary compromise to ensure reliable detection.

In accordance with the project requirements, we integrated a Cyclic Redundancy Check (CRC) mechanism to verify the integrity and correctness of the transmitted data. This design mandates that the receiving end, upon successfully detecting the preamble, must first validate the CRC checksum before accepting the data, thereby safeguarding the accuracy of the received information.

Initially, we adopted Amplitude Shift Keying (ASK) as our modulation scheme. However, it became apparent that signal amplitude is highly susceptible to severe attenuation during propagation through the air and is considerably vulnerable to interference from ambient noise. Therefore, while working on Task 4—which specifically demanded 100% data accuracy—we abandoned ASK and transitioned to Frequency Shift Keying (FSK). This alternative approach, which relies on

to Frequency Shift Keying (FSK). This alternative approach, which relies on variations in audio frequency to encode data, demonstrated superior performance in airborne transmission environments affected by noise. To further guarantee the target of 100% accuracy, we also implemented an Error Correction Code (ECC) mechanism, which allowed for the recovery and repair of partially corrupted data segments.

The key accomplishments of our implementation include:

- A physical layer based on FSK modulation.

- A CRC polynomial defined as $x^8 + x^2 + x + 1$ (corresponding to hexadecimal 0x07).

- Integration of ECC for enhanced error resilience.

- A set of supporting parameters derived and fine-tuned through extensive experimentation.

Throughout the development process, we encountered numerous challenges that extended beyond purely software-related issues to encompass significant hardware limitations. For instance, the built-in audio hardware of standard computers proved substantially less effective for this project compared to externally connected audio interfaces. Similarly, microphone placement—specifically, positioning the microphone at an angle to the speaker rather than directly facing it—resulted in markedly inferior performance. Although these hardware-related factors were not initially given due consideration, they later proved to have a profound impact on the system's overall effectiveness. This oversight ultimately led to considerable time investment before we fully recognized and addressed these critical issues.

Project 2

Project 2 moved the physical layer from air to cable. In principle the wire made things easier — signals became cleaner and more repeatable — but in practice new hardware quirks and timing issues emerged that forced us to re-think both the electrical setup and the code.

At the hardware level we ran into a few recurring problems. Different laptops and power adapters produced small DC offsets and occasional hum; long cable runs and nearby electronics introduced sporadic noise. Worse, ground differences between machines sometimes produced non-zero potentials that biased the

received waveforms and in rare cases risked damaging equipment. Our mitigation was three-fold: prefer directly grounded machines when possible, recommend unplugging power adapters during experiments, and keep the cable layout tidy and away from high-power adapters. We also pushed for short, shielded cables and used the mixer as a hub (rather than ad-hoc wiring) to improve signal consistency.

On the software side the biggest surprises came from timing. The audio stack and the USB sound card introduce measurable latency and occasional sample drops; this makes carrier-sensing and collision detection via the sound card unreliable. We therefore moved from optimistic carrier-detect designs to an ACK-based flow: every data block is framed, CRC-protected, sent, and retransmitted until an ACK arrives or a retry limit is reached. To make this practical we tuned buffer sizes and timers: increasing the audio buffer reduced underruns at the cost of higher latency, while a modest retransmit timeout (tuned to the ASIO/USB round-trip characteristics) struck the best balance between responsiveness and robustness.

Specific coding-level fixes we applied:

Framing and checksums: we standardized a compact frame format with a short preamble, length, flags, payload and CRC so corrupted data could be detected and discarded early.

Line coding: to avoid long runs of identical symbols (which caused DC bias and hardware issues) we adopted a transition-guaranteeing code (8B10B-like behavior) so the physical line always toggles often enough to prevent drift.

ACK state machine: the MAC thread implements a small state machine (Idle, TxPending, TxFrame, WaitingForAck, etc.). The sender sets a timeout after transmission, counts retransmissions, and raises an error after a maximum retry count. The receiver immediately replies with an ACK for good frames; corrupted frames are ignored.

Concurrency: audio I/O runs in the real-time callback; we decoupled packet processing by copying audio samples into thread-safe queues protected by lightweight locks so parsing and MAC logic run in worker threads and do not block audio.

Task 1 (Cable connection) challenges and solutions

Challenge: achieving a reliable raw transfer under a 20-second completion

challenges achieving a reliable raw transfer under a 20-second completion window for a 6,250-byte binary file when the audio hardware occasionally drops samples.

What we did: line coding and robust framing reduced bit errors, and we added a small retransmit window at the link layer to recover from occasional lost samples. On the hardware side we recommended unplugging power adapters and using the mixer as a hub to avoid ground issues; on the software side we increased the audio buffer slightly during transfers to avoid ASIO underruns and logged buffer underrun events so the TA could correlate failures with system load.

Task 2 (Acknowledgement) challenges and solutions

Challenge: collisions and mis-ordered events due to audio latency; the sender could even see its own ACK because driver delays lead to round-trip times on the order of 15–20 ms.

What we did: we implemented a simple ACK protocol with a clear MAC header that distinguishes DATA and ACK frames and includes receiver IDs so the sender can ignore echoes of its own ACKs. The MAC keeps a retransmit counter and a timeout; if the ACK doesn't arrive within the timeout the frame is retransmitted. We tuned the timeout to be comfortably larger than the measured audio round-trip latency, and we provided a link-error signal when retransmits exceed the limit. To help debugging we added console tracing of state transitions and retransmit counts so failures were easy to diagnose.

Overall, Project 2 is an exercise in practical trade-offs: line coding and framing handle the electrical pathology, careful buffer and timer tuning tame the audio stack, and a small ACK-based MAC gives the reliability needed for higher-layer tests.

Project 3

Project 3 aimed to implement some fundamental functions of the network layer. Task 1 required the realization of a ping command over an audio-based local area network, while Task 2 extended this requirement by adapting the ping command to operate across real-world networks.

Building upon the functionalities developed in Project 2, we enhanced the existing packet structure by incorporating MAC addresses and IPv4 addresses. The addition of these data fields inevitably increased the overall length of each

.....

packet, which subsequently led to a noticeable decline in packet reception success rates. In an effort to improve accuracy, we implemented several refinements to the preamble detection mechanism. The most effective version of our code successfully reduced the packet loss rate to approximately 50%. However, this performance still fell significantly short of the project's requirements. Due to issues such as poor time management, the accumulation of various minor coding errors, and other unforeseen challenges, we were unfortunately unable to complete Project 3 before the deadline.

Project 4

Project 4 asks us to extend the virtual network so it can resolve names and reach standard services: in practice that meant adding DNS and HTTP capability and upgrading NODE2's NAT to forward both UDP and TCP traffic. We configured NODE1 to route DNS and target traffic through the virtual interface and implemented a local resolver on NODE2 that answers NODE1's requests and, when needed, performs stepwise queries to external name servers on behalf of NODE1.

Difficulties: matching external DNS behavior, handling UDP timeouts and NAT translation races, and the subtle interactions between crafted TCP packets and the host OS networking stack.

How we fixed it: we used a small UDP client wrapper for reliable send/receive with timeouts, added a simple cache to reduce repeated lookups, and iterated on NAT corner-cases (port reuse and race conditions) until behavior matched expectations. For TCP tests we used helper processes to bind ports so replies would be captured correctly, which resolved the tricky handshake/capture interaction.

Overall, Project 4 ties together the earlier layers: the PHY and frame transport remain the same, while the DNS and NAT logic add application-level functionality that lets NODE1 resolve names and reach external services through NODE2.

In Project 4 the DNS subsystem is organized around a small resolver on NODE1 and a local recursive resolver on NODE2. NODE1 formats standard DNS request messages and delivers them to NODE2 inside the project's framed transport; those frames carry the raw DNS bytes and are decoded by the receiver.

On receipt, NODE2 first consults a synchronized cache to answer quickly when possible. If the name is not cached, NODE2 performs iterative resolution on

possible. If the name is not cached, NODE2 performs iterative resolution on behalf of NODE1: it sends the DNS request to external DNS servers using UDP, waits with a bounded timeout, and, on timeout, retries or fails over to alternate servers. When a response arrives, NODE2 matches it to the outstanding request, extracts address records, caches useful answers, and then returns the reply to NODE1 inside a framed transport message.

We treat DNS messages as opaque binary blobs and simply carry them inside our frames; the framing and a CRC let us detect and drop any corruption introduced by the PHY so the resolver only sees well-formed packets. For external resolution we relied on UDP because the DNS request/response pattern fits a connectionless model and keeps latency low — we add short receive timeouts and try alternate servers so an upstream outage doesn't hang the whole stack. To avoid repeating work, NODE2 keeps a small, thread-safe cache; access is synchronized so multiple simultaneous queries don't clash. When things go wrong we write human-readable logs and occasional hex dumps to the console to help trace malformed or truncated replies and we defensively reject suspicious frames. We deliberately left TCP fallback for truncated DNS replies and advanced features like DNSSEC as future improvements.

These choices keep the resolver simple and observable while fitting cleanly into the project's framed transport: frames carry DNS bytes between nodes, NODE2 acts as the recursive resolver with caching and UDP-based lookups, and the results are returned to NODE1 for use by ping, HTTP checks, and other higher-layer tests.